

AI & Data Science Track – Round 4
System Analysis & Design

Advanced Food Image Generation System

Text-to-Image Generation for Restaurant & Food Delivery Platforms

Team Members

Omar AbdelNasser (Team Lead) • Mohamed Tarek • Marcelino Maged • Verina Antonious • Momen
Mohamed

Section 1: Problem Statement, Use Cases & Architecture

1.1 Problem Statement

Restaurant owners and small food businesses frequently lack high-quality photographs of their menu items. Professional food photography is expensive, time-consuming, and inaccessible to small vendors — yet visual content is one of the most critical factors driving customer purchase decisions on food delivery platforms such as Talabat, Uber Eats, and similar services. Studies consistently show that menu items with appealing images receive significantly more orders than those without.

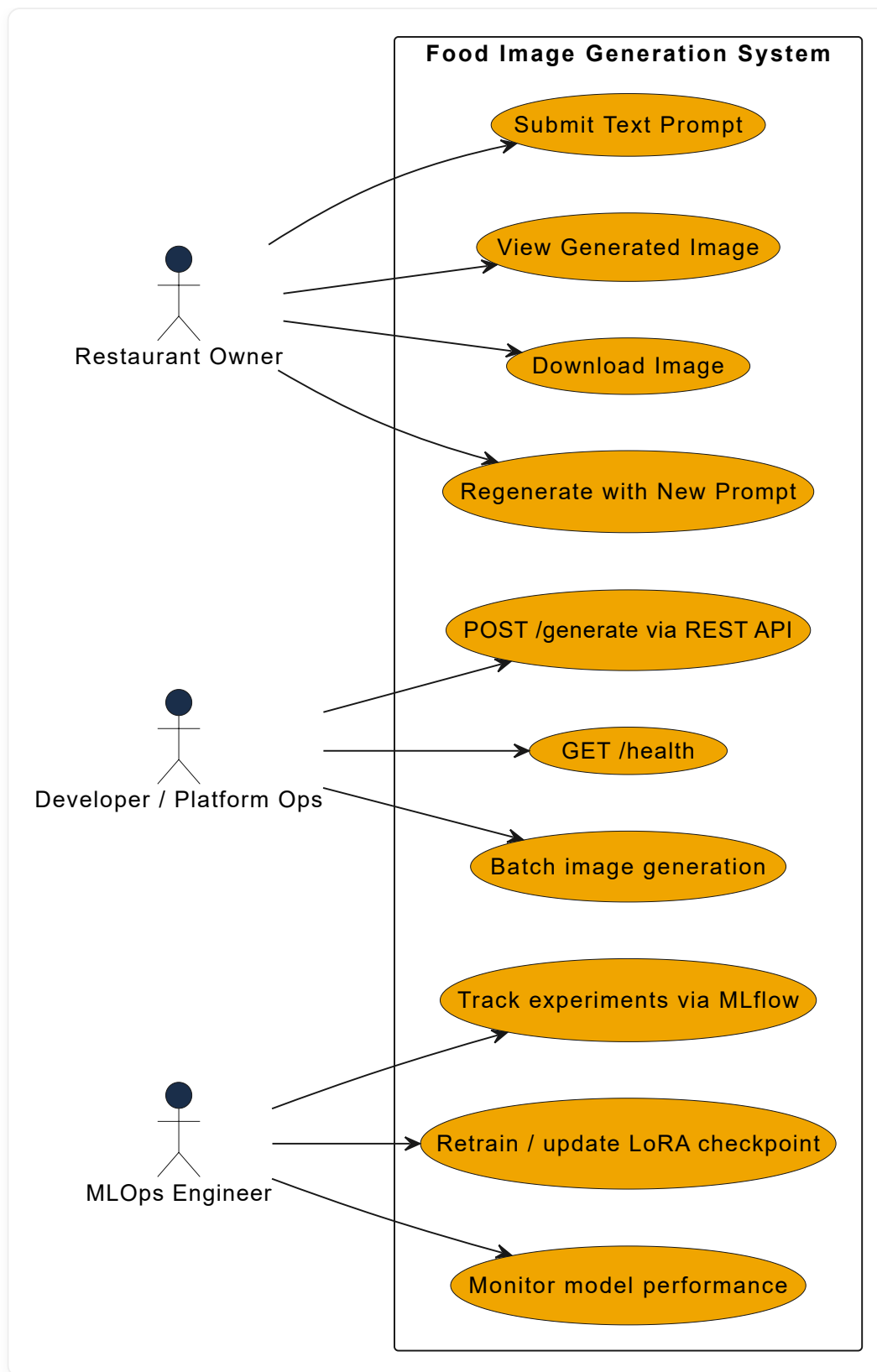
This project addresses the problem by building an AI-powered text-to-image generation system that allows a restaurant owner to type a natural-language description of a food item and receive a photorealistic image within seconds — at zero photography cost. The system leverages a LoRA fine-tuned Stable Diffusion model conditioned on CLIP text embeddings, serving requests through a FastAPI REST endpoint.

1.2 Project Objectives

- Curate a high-quality food image dataset from COCO using CLIP-based relevance scoring, retaining 10,570 clean image-caption pairs across 20 food categories.
- Fine-tune Stable Diffusion v1.5 using LoRA adapters targeting cross-attention layers, specializing the model in photorealistic food image generation.
- Implement a complete inference pipeline with DDIM sampling and classifier-free guidance for high-fidelity, prompt-adherent image generation.
- Expose the model via a FastAPI REST API and a minimal web UI accessible to non-technical restaurant owners.
- Track all experiments with MLflow, containerize the system with Docker, and ensure full reproducibility.

1.3 Use Case Diagram

The system has three primary actors: Restaurant Owner (end user via web UI), Developer/Platform Operator (API consumer), and MLOps Engineer (model management). The following diagram shows their interactions with the system:



1.4 Use Case Descriptions

Use Case	Actor	Description	Precondition	Outcome
Submit Text Prompt	Restaurant Owner	User types a food description into the web form and clicks Generate	System is running; FastAPI is active	Image generation job is created
View Generated Image	Restaurant Owner	System displays the generated food image after processing	Prompt has been submitted	User sees a photorealistic food image
Download Image	Restaurant Owner	User downloads the generated image as PNG	Image has been generated	PNG file is saved locally
POST /generate	Developer	API client sends JSON with prompt field, receives PNG image in response	Valid API request with non-empty prompt	200 response with image bytes
GET /health	Developer	Check if the inference service is alive and the model is loaded	Service is deployed	200 JSON with status and model info
Track Experiments	MLOps Engineer	MLflow logs training run with parameters, loss curves, and LoRA checkpoint	Training loop is running	Experiment recorded in MLflow UI
Update LoRA Checkpoint	MLOps Engineer	New LoRA weights are loaded into the inference service without restarting	New checkpoint is available	Model serves improved outputs

1.5 Functional & Non-Functional Requirements Summary

Functional Requirements

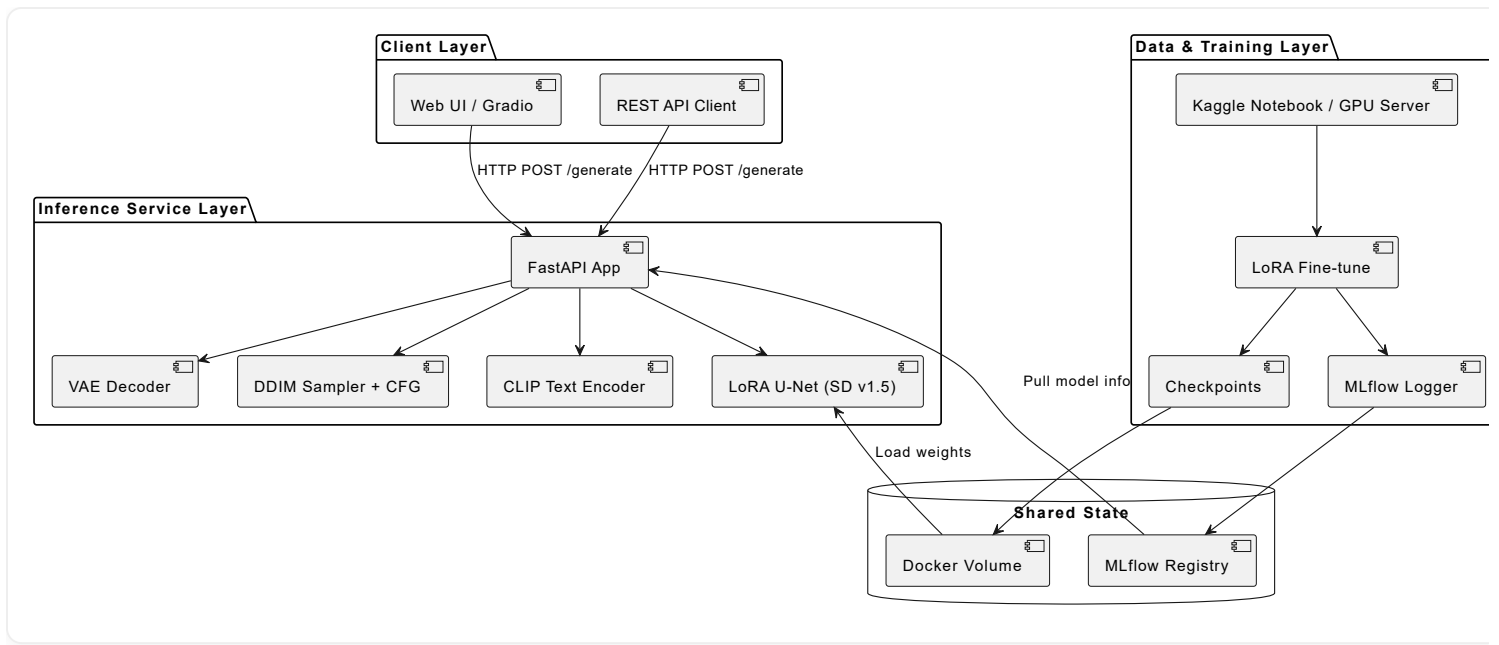
ID	Requirement	Priority
FR-01	Filter COCO dataset using keyword matching and CLIP relevance scoring (threshold 0.35) to build a clean food-specific training set.	Must Have
FR-02	Preprocess images to 512x512 with normalization [-1,1] and augmentation (flip, color jitter).	Must Have
FR-03	Encode prompts using frozen CLIP ViT-B/32 text encoder into conditioning vectors.	Must Have
FR-04	Apply LoRA adapters (r=16, alpha=16) to U-Net cross-attention layers and fine-tune on food dataset.	Must Have
FR-05	Run DDIM sampling (50 steps) with classifier-free guidance (scale 7.5-9.0) for inference.	Must Have
FR-06	Expose POST /generate FastAPI endpoint returning PNG image from text prompt.	Must Have
FR-07	Support negative prompts at inference to suppress cartoon/blurry styles.	Should Have
FR-08	Log all training runs to MLflow (params, loss, FID score, LoRA checkpoint).	Must Have
FR-09	Package full inference service in Docker container.	Must Have
FR-10	Compute FID and Inception Score on held-out validation set.	Should Have

Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	Image generation < 15 seconds on T4 GPU with 50 DDIM steps.
NFR-02	Reliability	FastAPI uptime > 95% during demo session.
NFR-03	Scalability	System deployable on any CUDA GPU with >= 8GB VRAM via Docker.
NFR-04	Usability	Non-technical user generates image in <= 2 UI interactions.
NFR-05	Reproducibility	All runs reproducible via fixed seed + MLflow-logged hyperparameters.
NFR-06	Security	Input prompts validated and truncated at 77 CLIP tokens.
NFR-07	Portability	Python 3.10+, all deps pinned in requirements.txt.
NFR-08	Ethics	Training data under CC BY 4.0; base model under CreativeML RAIL-M license.

1.6 Software Architecture

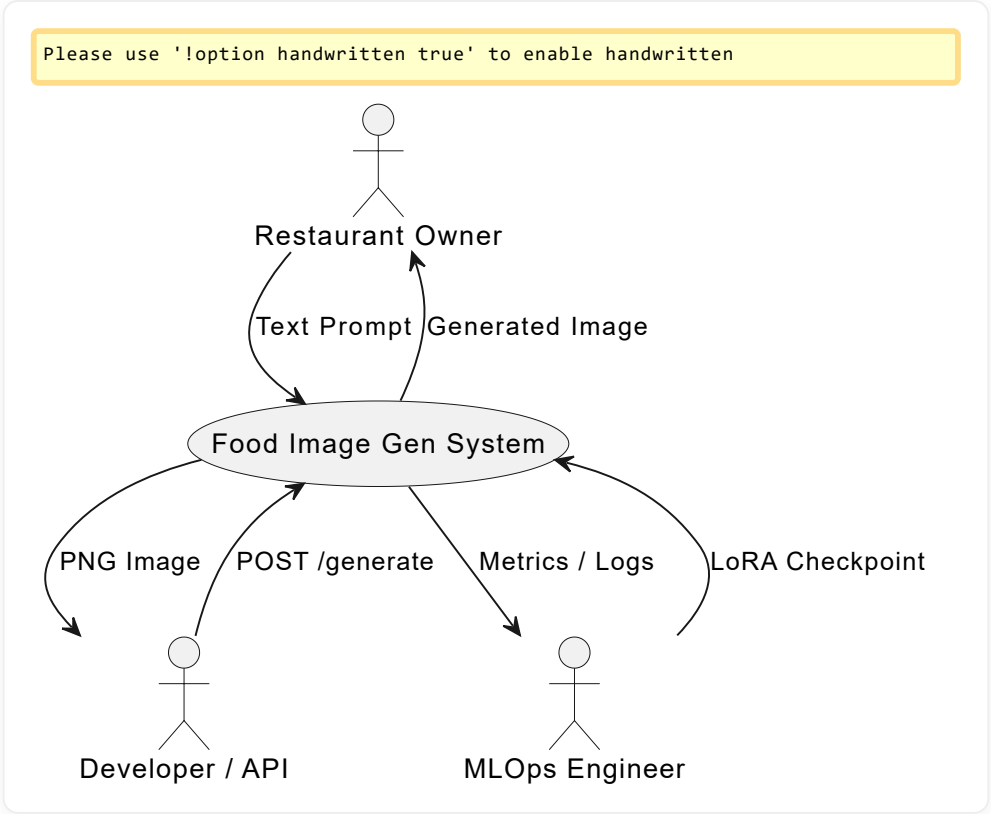
The system follows a Microservices-inspired architecture with three loosely coupled layers: the Data & Training Layer, the Inference Service Layer, and the Client Layer. These communicate via REST APIs and share state through the MLflow model registry and Docker volume mounts.



Section 2: Data Flow & System Behavior

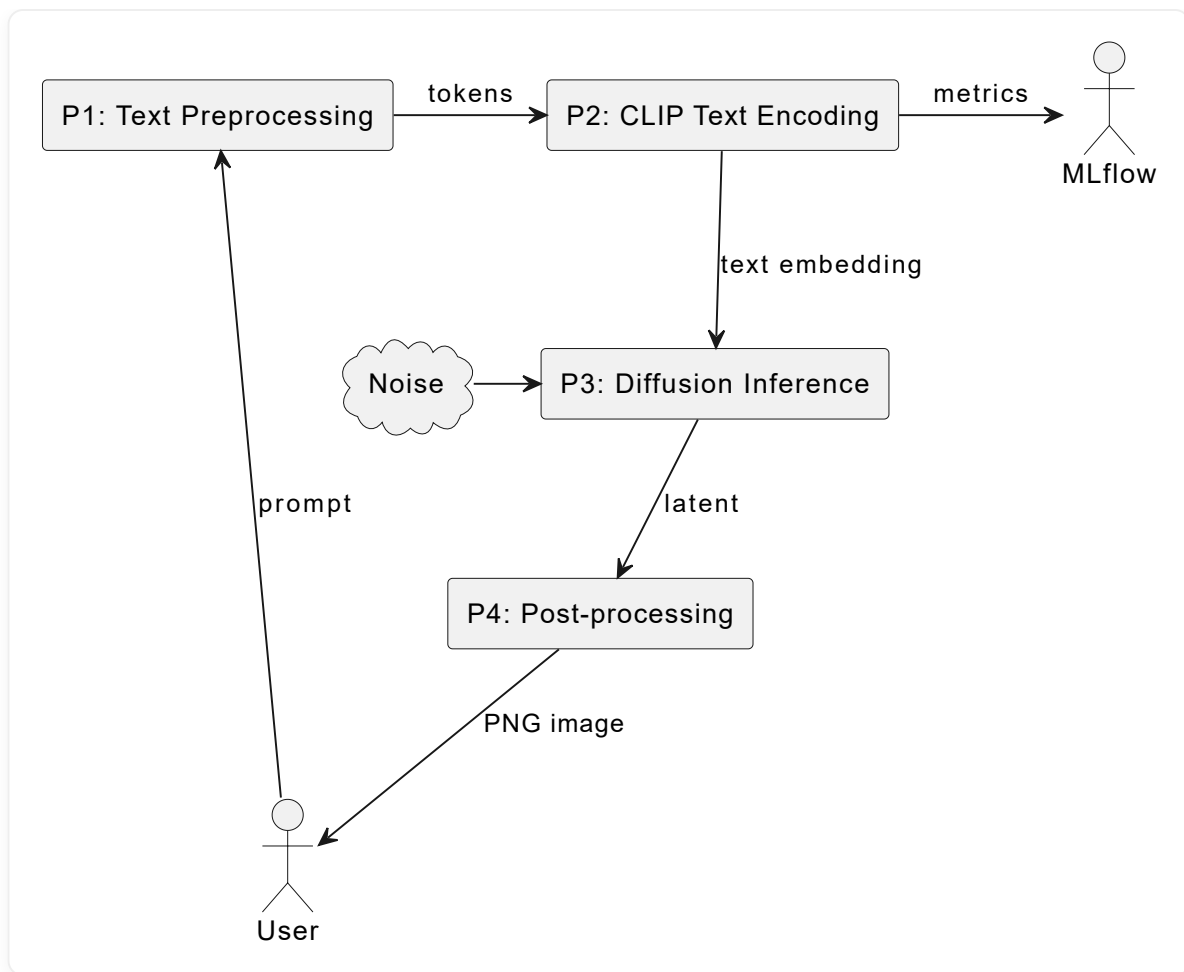
2.1 Data Flow Diagram — Level 0 (Context Diagram)

The context diagram shows the system as a single process with its external entities and the data flows between them.



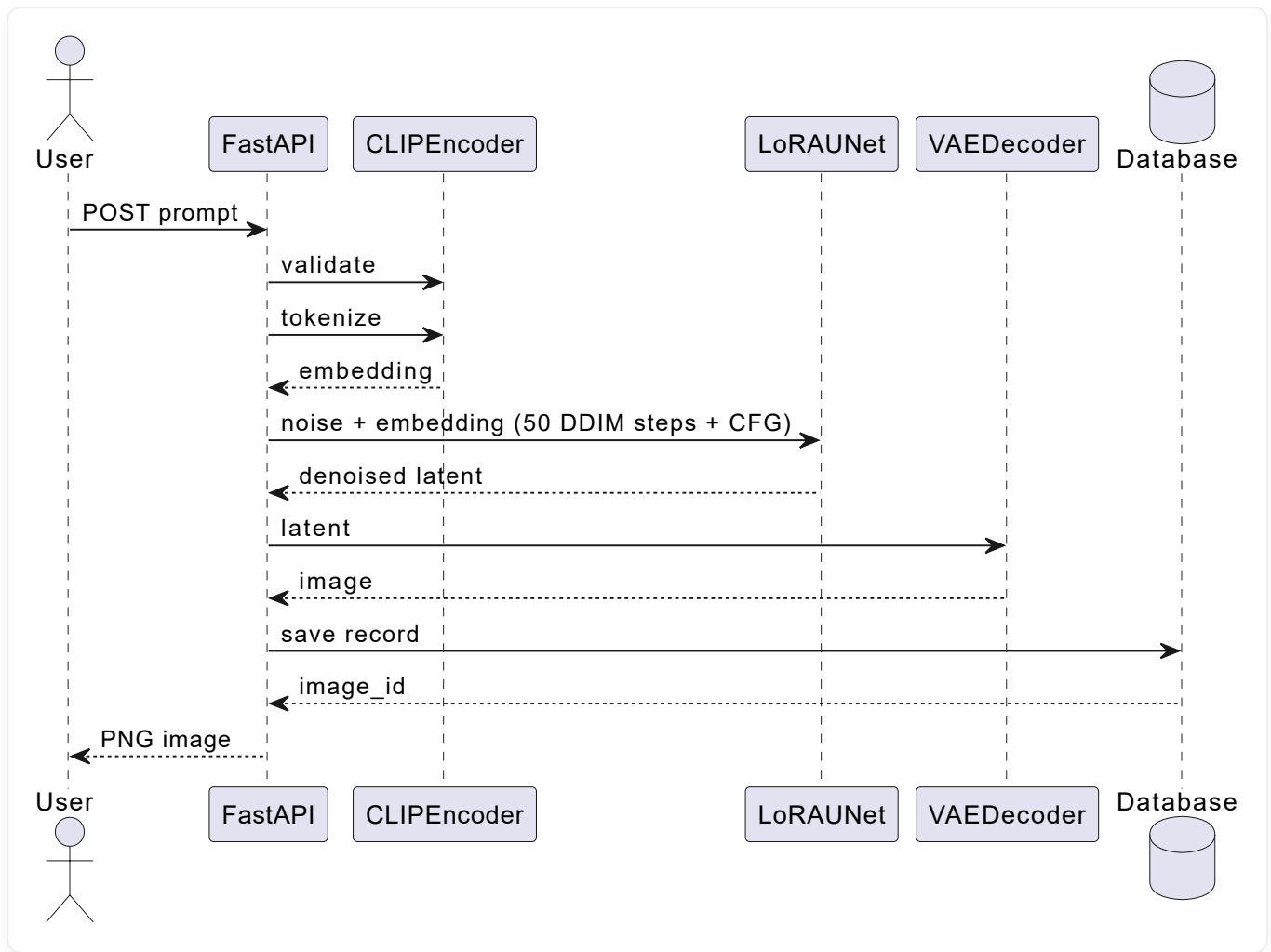
2.2 Data Flow Diagram — Level 1 (Detailed)

The Level 1 DFD decomposes the system into four main processes:



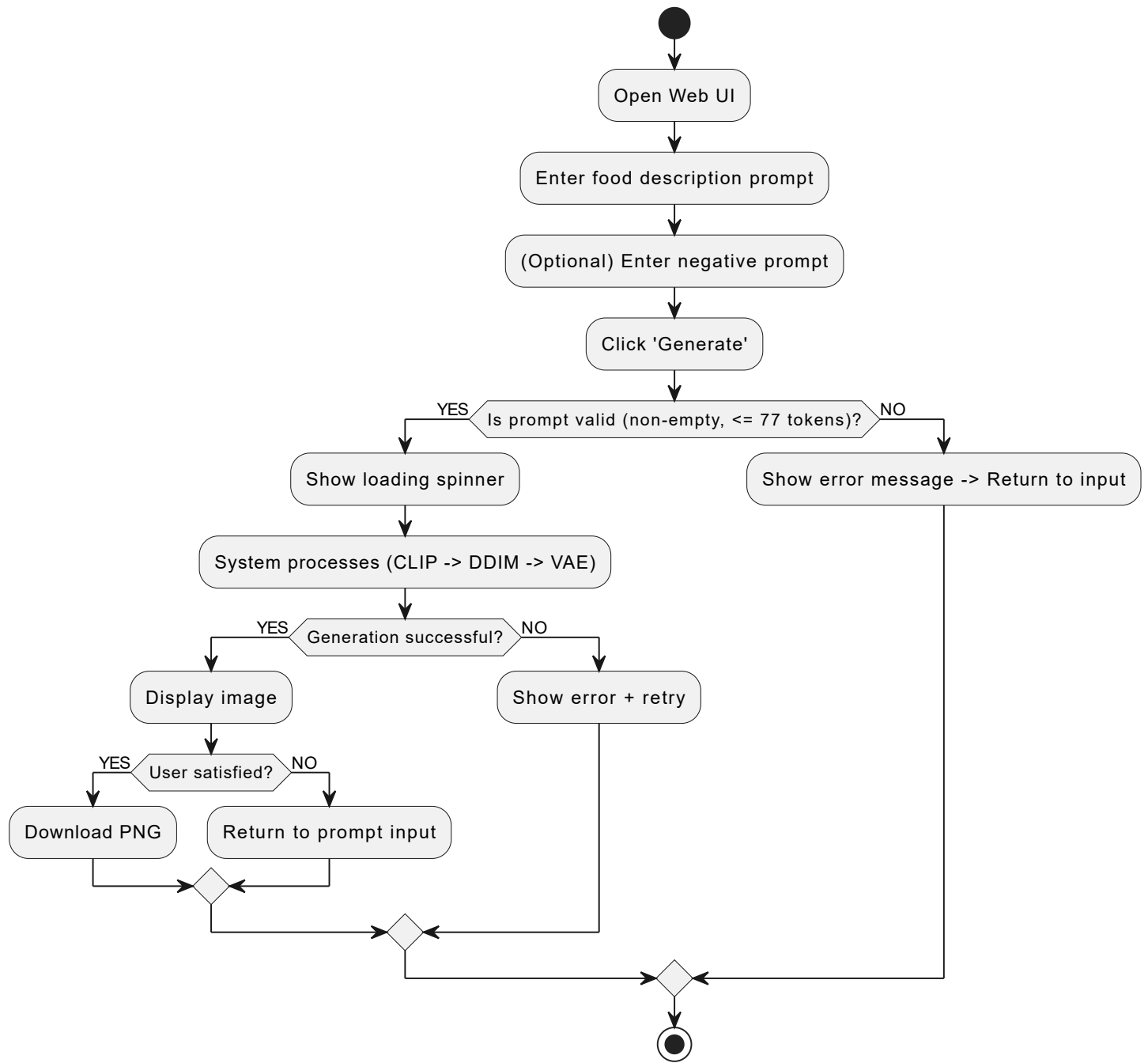
2.3 Sequence Diagram — Image Generation Flow

The sequence diagram shows the interaction between components during a single image generation request.



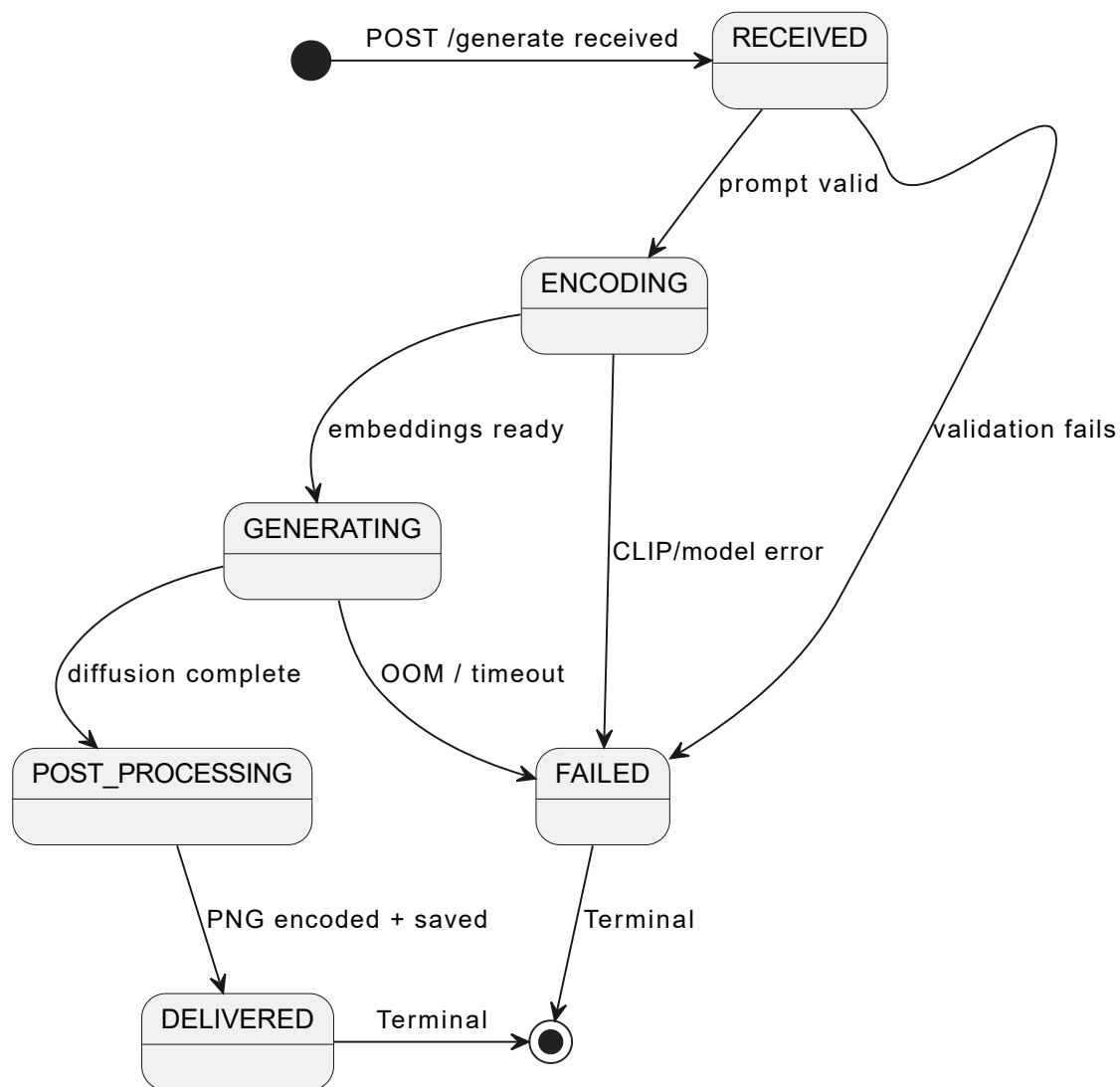
2.4 Activity Diagram — User Workflow

The activity diagram shows the complete workflow from a restaurant owner opening the web UI to downloading the final image.



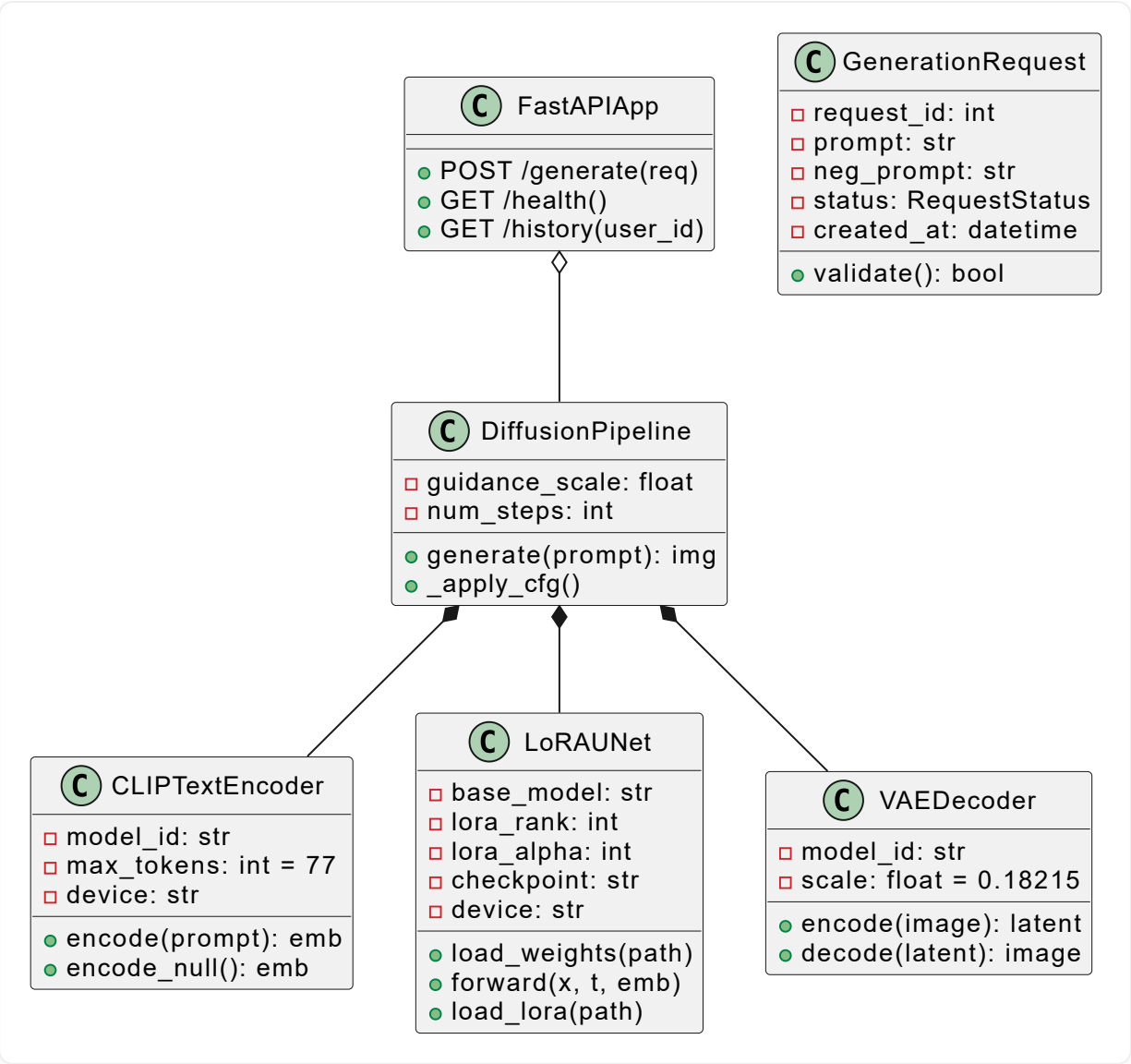
2.5 State Diagram — GenerationRequest Lifecycle

A GenerationRequest object transitions through the following states from submission to completion or failure.



2.6 Class Diagram

The following class diagram shows the main Python classes in the system, their attributes, methods, and relationships.



Section 3: UI/UX Design & Prototyping

3.1 Design Principles

- **Professionalism:** The interface is designed to look and feel like a polished SaaS product — clean card layouts, consistent spacing, and a refined visual hierarchy that builds trust with restaurant owners and platform operators alike.
- **Simplicity of use despite visual richness:** Although the UI is visually professional, the primary action (generating an image) still requires exactly two steps: enter a description, click Generate. Complexity is in the aesthetics, not the workflow.
- **Brand consistency:** The design uses a food-industry-appropriate color palette — deep navy and warm amber — conveying appetite appeal and reliability simultaneously.
- **Speed feedback:** A polished animated loading state with a progress indicator and elapsed time prevents users from thinking the system has frozen during the 10–15 second generation process.
- **Accessibility:** Minimum contrast ratio of 4.5:1 for all text. All interactive elements are keyboard-navigable. Generated images include alt text derived from the prompt.

3.2 UI/UX Guidelines

Element	Specification
Primary font	Inter or system sans-serif, 16px base size — clean and modern
Heading font	Inter Bold, 24–32px — strong visual hierarchy
Primary color	#1A2E4A (deep navy) — professional, trustworthy
Accent color	#F0A500 (warm amber) — food-themed, used for CTA buttons and highlights
Background	#F4F6F9 (light blue-grey) — softer than pure white, reduces eye strain
Card background	#FFFFFF (white) — elevated cards with subtle box-shadow
Text primary	#1A1A2E (near-black) — high contrast body text
Text secondary	#6B7280 (medium grey) — labels, metadata, helper text
Error / Success	Error: #D93025 (red) Success: #1E8E3E (green)
Button style	Rounded corners (8px), amber fill, navy text, hover state darkens by 10%, minimum 44px touch target
Cards	White background, 12px border radius, subtle box-shadow (0 2px 8px rgba(0,0,0,0.08))

3.3 Wireframe — Screen 1: Home / Prompt Input Page

FoodGen AI

Docs

API

Sign In

Generate Professional Food Photography
Instantly. Powered by AI.

Describe your dish...
e.g. 'Grilled salmon with lemon and herbs'

Negative prompt (optional) ▼

Generate Image

 ← amber button

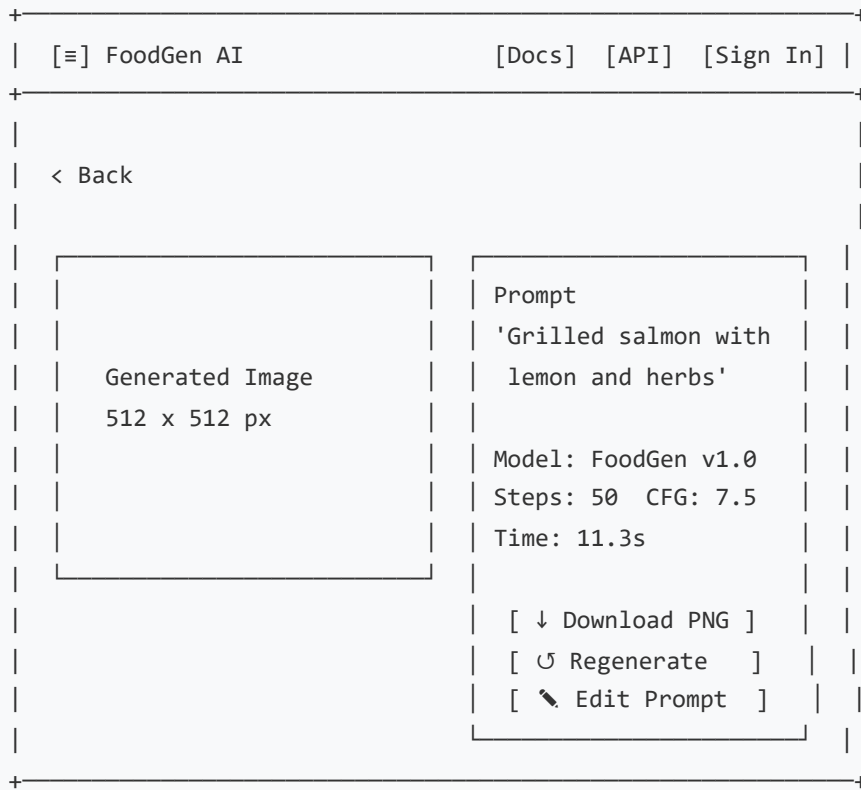
Quick examples

Pizza Margherita

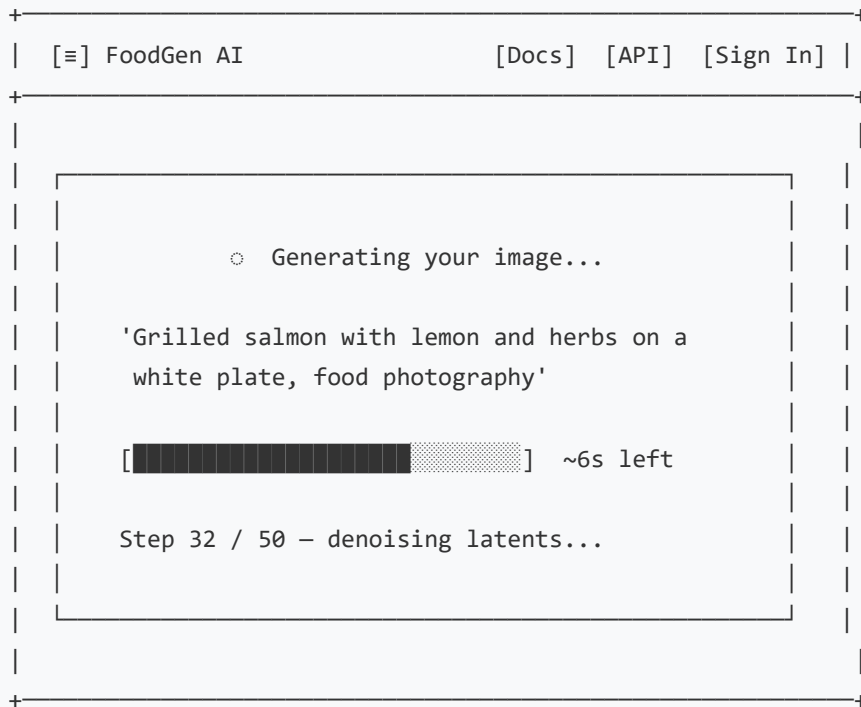
Birthday Cake

Ramen Bowl

3.4 Wireframe — Screen 2: Generation Results Page



3.5 Wireframe — Screen 3: Loading State

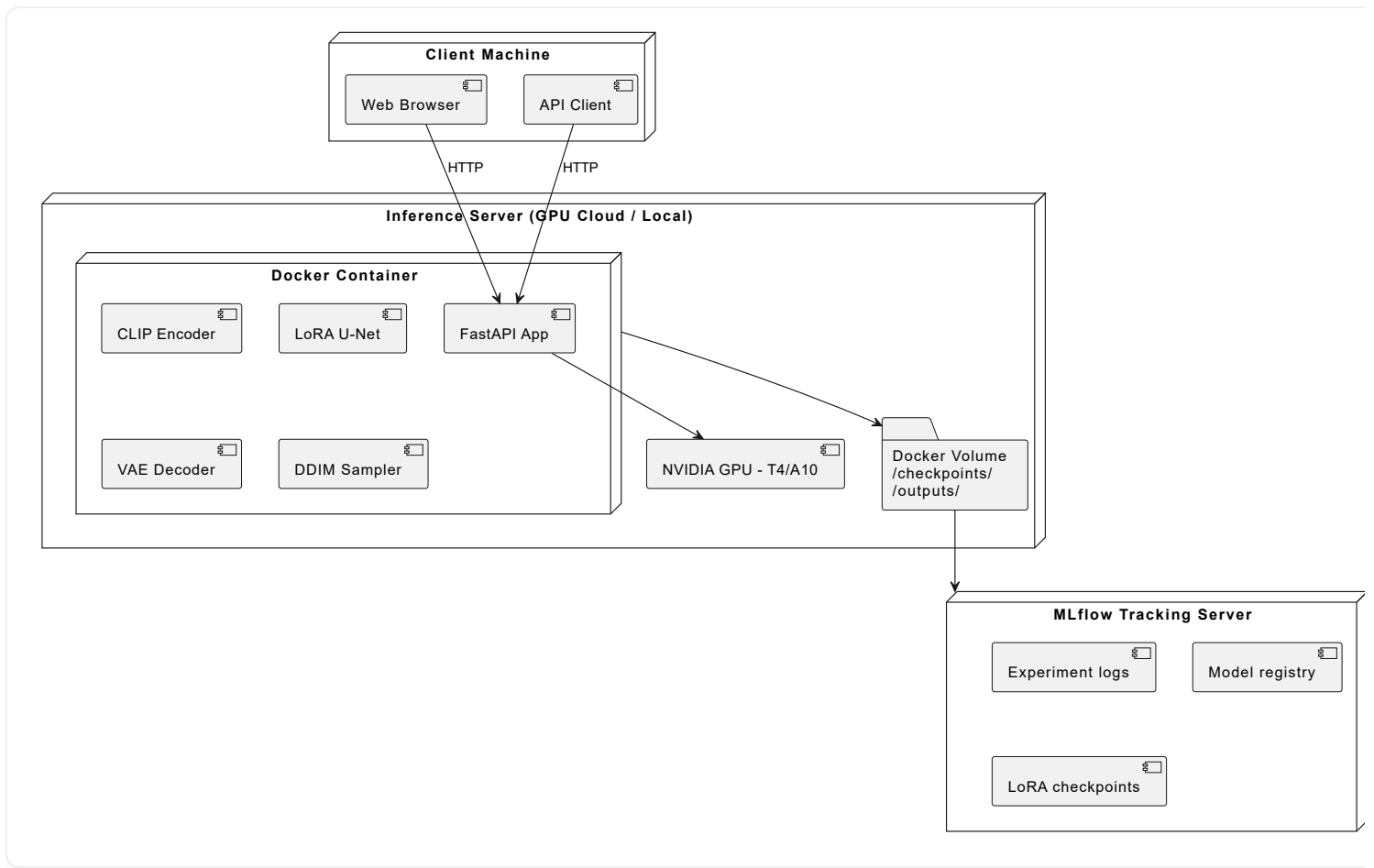


Section 4: System Deployment & Integration

4.1 Technology Stack

Layer	Technology	Purpose
ML Framework	PyTorch 2.x	Core deep learning framework for training and inference
Diffusion Library	HuggingFace Diffusers	Stable Diffusion pipeline, schedulers, VAE, U-Net
LoRA Adapters	PEFT (HuggingFace)	LoRA fine-tuning of U-Net attention layers
Text Encoding	CLIP (ViT-B/32)	Frozen text encoder for prompt conditioning
Training Compute	Kaggle T4 GPU (16GB)	GPU environment for LoRA fine-tuning
Backend API	FastAPI + Uvicorn	REST API server for inference serving
Web UI	Gradio	Minimal web interface for non-technical users
Experiment Tracking	MLflow	Logging parameters, metrics, and LoRA checkpoints
Visualization	TensorBoard	Training loss curves and image sample grids
Containerization	Docker + NVIDIA Container Toolkit	Reproducible deployment across environments
Dataset	HuggingFace Datasets (jxie/coco_captions)	Streaming COCO captions and images
Language	Python 3.10+	Primary programming language

4.2 Deployment Diagram



4.3 Component Diagram

FoodGen System

data_pipeline

filter.py

preprocess.py

embeddings.py

dataloader.py

api

main.py

endpoints.py

schemas.py

ui.py

model

lora.py

unet.py

vae.py

clip.py

sampler.py

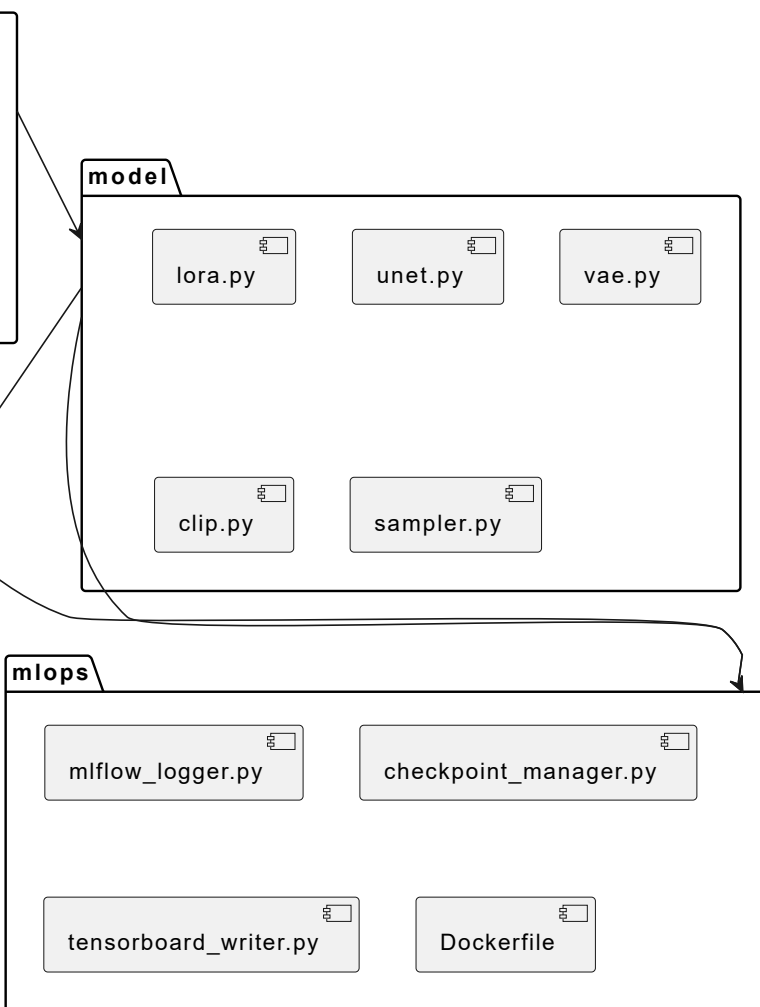
mlops

mlflow_logger.py

checkpoint_manager.py

tensorboard_writer.py

Dockerfile



Section 5: Additional Deliverables

5.1 API Documentation

Base URL: <http://localhost:8000> (development) | <https://api.foodgen.ai> (production)

Endpoint 1 — POST /generate

Field	Details
Method	POST
URL	/generate
Description	Submit a text prompt and receive a generated food image as PNG bytes.
Auth	API key in header: X-API-Key: <your_key>
Request body	<pre>{{ "prompt": "a slice of pepperoni pizza on a white plate", "negative_prompt": "blurry, cartoon, unrealistic", "guidance_scale": 7.5, "num_steps": 50 }}</pre>
Success response	200 OK — image/png binary content
Error responses	400 Bad Request (empty or too-long prompt) 422 Unprocessable Entity (invalid field types) 500 Internal Server Error (model/GPU failure)
Example curl	<pre>curl -X POST http://localhost:8000/generate \ -H "Content-Type: application/json" \ -H "X-API-Key: your_key" \ -d '{"prompt": "chocolate birthday cake"}' \ --output cake.png</pre>

Endpoint 2 — GET /health

Field	Details
Method	GET
URL	/health
Description	Check service status and confirm model is loaded.
Auth	None required
Success response	200 OK — {"status": "ok", "model": "FoodGen-LoRA-r16-v1", "device": "cuda"}

5.2 Testing & Validation Plan

Unit Tests

ID	Component	Test Description	Expected Result
UT-01	CLIPTextEncoder	Encode a sample prompt and check output shape	Tensor shape (1, 768)
UT-02	CLIPTextEncoder	Encode prompt > 77 tokens; verify truncation	No error; shape still (1, 768)
UT-03	VAEDecoder	Encode image → latent → decode → compare shape	Output shape matches input (3,512,512)
UT-04	DDIMSampler	Run 5-step sample with dummy embedding; verify output shape	Tensor shape (1,3,512,512)
UT-05	FastAPI /health	GET /health when model loaded	200 OK with JSON status
UT-06	FastAPI /generate	POST empty prompt	400 Bad Request
UT-07	FastAPI /generate	POST valid prompt	200 OK with PNG bytes
UT-08	GenerationRequest	Validate request with missing prompt field	ValidationError raised

Integration Tests

ID	Description	Expected Result
IT-01	Full pipeline: text prompt → CLIP → DDIM → VAE → PNG file	Valid PNG file generated, non-blank, size > 10KB
IT-02	FastAPI end-to-end: POST request → model inference → response	PNG returned in < 20 seconds
IT-03	MLflow logging: run training step and verify log entry exists	Run appears in MLflow UI with correct params
IT-04	Docker: build container, run health check inside container	Container starts, /health returns 200

User Acceptance Testing (UAT)

Scenario	Steps	Pass Criteria
Restaurant owner generates pizza image	1. Open UI 2. Type 'pepperoni pizza on white plate' 3. Click Generate 4. View result	Image generated in < 15s; visually shows a pizza; user can download PNG
Developer calls API	1. Send POST /generate with valid JSON 2. Save response as PNG 3. Open PNG	Valid PNG file received; image matches food description
Invalid input handling	1. Submit empty prompt 2. Submit 200-word prompt	Friendly error message shown for both cases; no system crash

5.3 Deployment Strategy

Development Environment

- Kaggle Notebooks with T4 GPU for training and prototyping.
- Local Python environment with CPU inference for API development and testing.

Production Deployment

- Docker image built with NVIDIA CUDA 11.8 base, all Python dependencies pinned in requirements.txt.
- Model weights (LoRA checkpoint ~50MB) mounted as a Docker volume — allowing hot-swapping without container rebuild.
- FastAPI served via Uvicorn with 1 worker (GPU is not multi-process safe without additional configuration).
- NGINX reverse proxy in front of Uvicorn for SSL termination and request queuing.

Scaling Considerations

- Horizontal scaling: multiple Docker containers behind a load balancer, each with its own GPU instance.
- Request queue: Redis queue between API gateway and inference workers to handle burst traffic without dropping requests.
- Model caching: CLIP encoder and VAE weights are loaded once at startup and kept in VRAM, only the LoRA U-Net weights are swappable.

Environment Variable	Description	Example
MODEL_CHECKPOINT	Path to LoRA weights directory	/checkpoints/lora_epoch_5
GUIDANCE_SCALE	Default CFG guidance scale	7.5
NUM_STEPS	Default DDIM steps	50
MAX_PROMPT_LENGTH	Max prompt token length	77
MLFLOW_TRACKING_URI	MLflow server URI	http://mlflow:5000
API_KEY_SECRET	Secret for API key validation	set via Docker secret